Password Analyzer Project Report

Project Members

Student Name	***Percentage***
Razan Alshahrani	25%
Team member 1	25%
Team member 2	25%
Team member 3	25%

1. Introduction

This project is a Password Analyzer developed using Java. It ensures that user-provided passwords meet strict security standards. If a password fails validation or contains weak patterns, the system generates a strong password that meets all implemented criteria.

2. Password Requirements

- Password length must be between 8 to 12 characters
- Must contain: one uppercase letter, one lowercase letter, one digit, one symbol
- Must not be similar to email
- No predefined common weak words
- No four or more identical consecutive characters or predefined sequential patterns

3. Design and Implementation

The system uses these Java components:

1. ***main Class (Main Program)***



```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package passwordvalidator;
import java.util.Scanner;
import static passwordvalidator.PasswordValidator.isValidPassword;
import passwordvalidator.PasswordSuggestion;

public class main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner( source: System.in);


        System.out.print("Enter your email: ");
        String email = scanner.nextLine();


        System.out.print("\n The recommended password must: "
                + "\n1-Be between 8 & 12 characters"
                + "\n2-Contains at least: "
                + "\n -one uppercase letter"
                + "\n -one lowercase letter"
                + "\n -one digit "
                + "\n -one symbol "
                + "\n3-Not be similar to email "
                + "\n \nEnter your password: ");
        String password = scanner.nextLine();
```



```
        boolean valid = isValidPassword(password, email);
        boolean weak = PasswordWeaknessChecker.weakPassword( pass: password);

        // Provide feedback to the user and Suggesting a secure password
        if (valid && !weak) {
            System.out.println("Strong password!");
        } else {
            System.out.println("Weak password\nPlease use a different one!");

            String suggested_password = PasswordSuggestion.StrongPassword(email);
            System.out.println("Here's a recommended strong password: " + suggested_password);
        }

        scanner.close();
    }
}
```

Purpose:

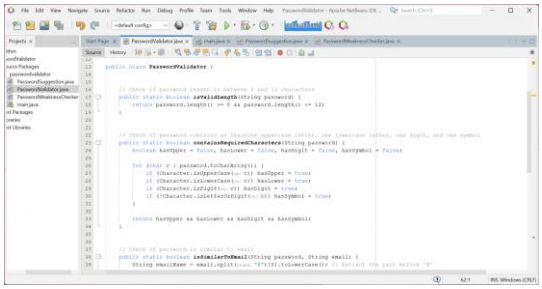
Acts as the starting point of the program and manages the flow of the password validation process.

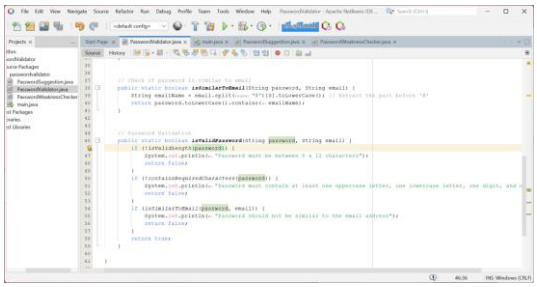
What it does:

- Prompts the user to enter their email and password.
- Displays the password length and required character types, along with the email similarity rule.

- Calls isValidPassword() from PasswordValidator to check if the password follows the structure rules.
- Calls weakPassword() from PasswordWeaknessChecker to identify any weak patterns.
- If the password is valid and strong, it prints a success message.
- If the password is invalid or weak:
- It prints why the password failed.
- It calls StrongPassword(email) from PasswordSuggestion to suggest a secure alternative

2.PasswordValidator Class





Purpose:

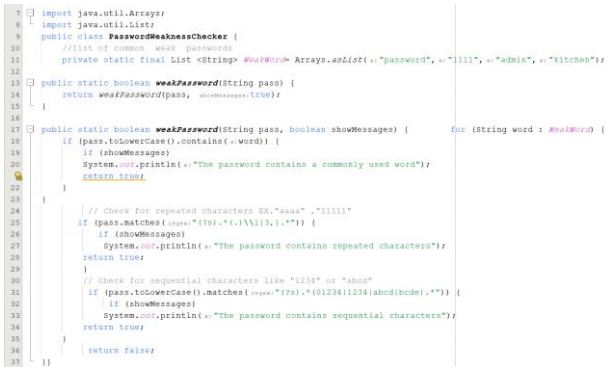
Validates the format and structure of the password to ensure it meets security criteria.

What it checks:

- isValidLength(password)
- Verifies that the password length is between 8 and 12 characters.
- containsRequiredCharacters(password)
- Confirms that the password contains:
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special symbol
- isSimilarToEmail(password, email)

- Ensures the password does not contain the email's username (the part before @).
- isValidPassword(password, email)
- Combines all above methods.
- If any rule fails, it prints the reason and returns false

3.**PasswordWeaknessChecker Class**



Purpose:

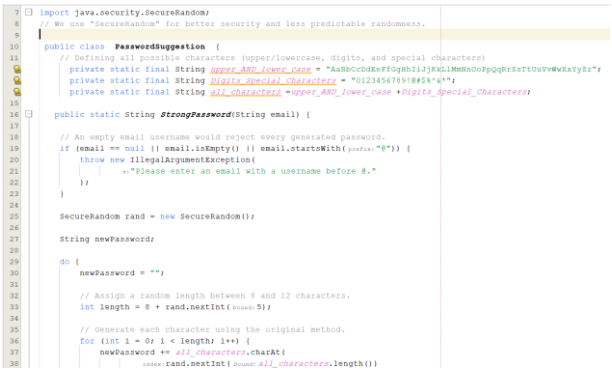
Checks if the password contains weak or predictable patterns.

What it detects:

- Common weak words like "password", "admin", "1111", or "kitchen".
- Detects four or more identical consecutive characters anywhere in the password.
- Detects the predefined sequences "01234", "1234", "abcd", and "bcde", ignoring letter case.
- weakPassword(String pass)
- Runs all checks.

The overloaded method weakPassword(String pass, boolean showMessages) allows the program to check generated passwords without displaying rejection messages.

4. **PasswordSuggestion Class**





Purpose:

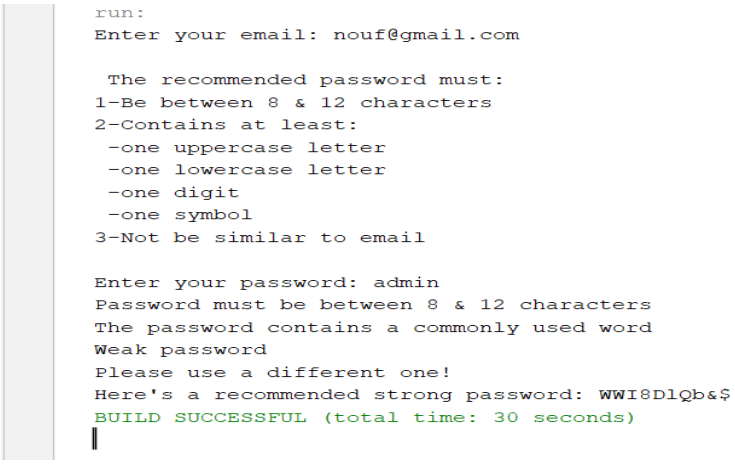
Generates a strong password when the user input is weak or invalid.

How it works:

- Uses SecureRandom to randomly select password characters.
- Selects a password length between 8 and 12 characters randomly.
- Mixes:
- Uppercase and lowercase letters
- Numbers
- Special characters (e.g., !@#$%^&*)
- Ensures that the password includes all required types of characters.
- StrongPassword(String email)
- Repeats password generation until the password contains all required character types, does not contain the email username, and passes the implemented weakness checks.

4. Sample Outputs

Case 1

Email Entered	nouf@gmail.com
Password Entered	admin
Issue Detected	The password is shorter than 8 characters and contains the common weak word “admin”. A strong alternative is suggested.
Output Screenshot	

```
run:
Enter your email: nouf@gmail.com

 The recommended password must:
1-Be between 8 & 12 characters
2-Contains at least:
 -one uppercase letter
 -one lowercase letter
 -one digit
 -one symbol
3-Not be similar to email

Enter your password: admin
Password must be between 8 & 12 characters
The password contains a commonly used word
Weak password
Please use a different one!
Here's a recommended strong password: WWI8DlQb&$
BUILD SUCCESSFUL (total time: 30 seconds)
```

Case 2

Email Entered	nouf@gmail.com
Password Entered	AAAAa1!x
Issue Detected	The password contains four consecutive identical characters (“AAAA”). It is rejected, and a strong alternative is suggested.

Output Screenshot

```
run:
Enter your email: nouf@gmail.com

 The recommended password must:
1-Be between 8 & 12 characters
2-Contains at least:
 -one uppercase letter
 -one lowercase letter
 -one digit
 -one symbol
3-Not be similar to email

Enter your password: AAAAa1!x
The password contains repeated characters
Weak password
Please use a different one!
Here's a recommended strong password: tIx2A#J*B
BUILD SUCCESSFUL (total time: 15 seconds)
```

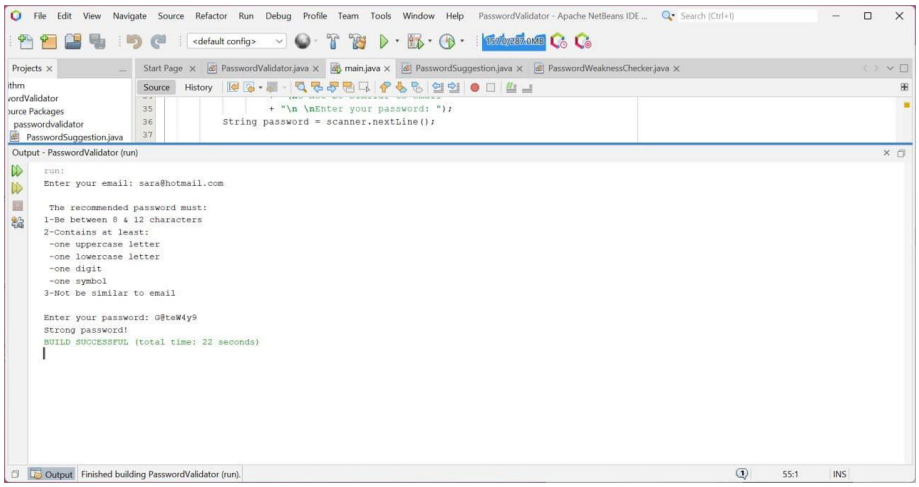
Case 3

Email Entered sara@hotmail.com

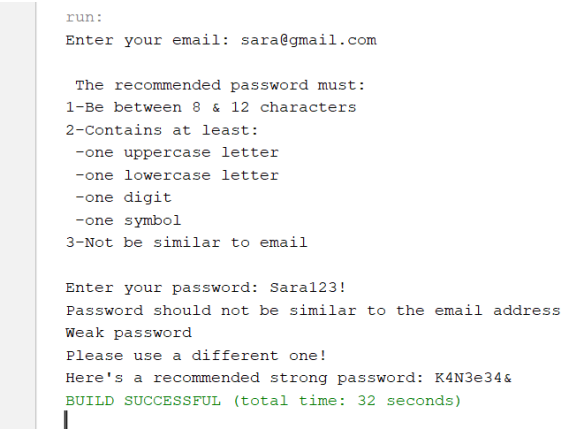
Password Entered G@tew4y9

Result: The password passes all implemented checks. The system displays “Strong password!”.

Output Screenshot



Case 4

Email Entered	sara@gmail.com
Password Entered	Sara123!
Result:	The password contains the email username “sara”, ignoring letter case. It is rejected, and a strong alternative is suggested.
Output Screenshot	

```
run:
Enter your email: sara@gmail.com

 The recommended password must:
1-Be between 8 & 12 characters
2-Contains at least:
 -one uppercase letter
 -one lowercase letter
 -one digit
 -one symbol
3-Not be similar to email

Enter your password: Sara123!
Password should not be similar to the email address
Weak password
Please use a different one!
Here's a recommended strong password: K4N3e34&
BUILD SUCCESSFUL (total time: 32 seconds)
```

5. Challenges and Solutions

Throughout the project, one of the trickiest parts was making sure the password checks worked exactly as they should—especially when trying to catch things like weak patterns or passwords that were too similar to the user’s email. It took some trial and error to get everything right. Breaking the code into smaller, focused parts really helped us stay organized and made the tough parts easier to handle. Working together and talking through our ideas also made a big difference when things got confusing.

6. Conclusion

Working on the Password Analyzer project gave us a chance to tackle a real-world security issue using what we’ve learned in Java. We built a system that doesn’t just check if a password follows certain rules—it also looks for common weaknesses and helps users choose stronger options. It was really rewarding to see how something as simple as checking a password can turn into a full-featured program with real impact.

Throughout the process, we learned a lot—not just about coding with Java, but also about how to think critically about user input, design clean and organized code, and create something that’s both secure and user-friendly. Breaking things down into clear modules

like validation, weakness checking, and password suggestions helped us stay organized and made the whole project easier to manage.

This project taught us how powerful programming can be when it's used to solve everyday problems. And more than that, it reminded us of the importance of good teamwork and communication.

Weak-word and sequence detection is limited to the predefined entries in the code.